

4-person detailed todo list for Modifai hackathon over 4 days

0

Done

76

Remaining

76

Total tasks

0%

Complete

P1 — Agent core

DAY 1 · FOUNDATION

Set up Bedrock tool-use boilerplate — one working round-trip call with a tool definition [ship it](#)

Define the Orchestrator agent's input schema: goal string + doc metadata → strategy JSON (intent, threshold, samples_per_chunk)

Write the Orchestrator system prompt — be specific about what decisions it must make and what format it must output

Implement basic retry logic: if strategy JSON is malformed, re-prompt once before failing

Share strategy JSON schema with P2 by EOD — this is the integration contract [sync](#)

0/5

DAY 2 · CRITIC AGENT

Build the Critic agent: takes one (instruction, input, output) sample, returns {verdict: accept|rewrite|reject, reason, rewritten_output?}

Write Critic system prompt — evaluate specificity, factual grounding, and format adherence; never hallucinate rewrites

Lock and publish Critic output JSON schema to the team — P1 cannot change this after Day 2 [blocker](#)

Batch mode: run Critic over 20 samples, return list of verdicts with aggregate stats (accept%, reject%)

Unit test: feed 5 deliberately bad samples, verify all get rejected or rewritten

0/5

DAY 3 · CURRICULUM AGENT

Build the Curriculum agent: reads Critic rejection reasons, identifies patterns, outputs a targeted generation prompt

Curriculum must produce at least 3 specific gap categories (e.g. 'lacks step-by-step reasoning', 'too vague on entities')

Wire Orchestrator → Critic → Curriculum into a single callable Python function that returns loop state

Test the full 3-agent chain end-to-end on real sample data — not mocks [blocker](#)

Document each agent's prompt, schema in and schema out — P4 needs this for README

0/5

DAY 4 · POLISH

Run full pipeline 3 times on the demo dataset — confirm Critic score improves each loop iteration

Add agent decision logging: every decision written to a structured JSON event stream for P3's dashboard

Edge case: handle empty Critic batch (all accepted first pass) gracefully — skip Curriculum, exit loop

Pair with P2 for 1 hour to fix any integration bugs before submission cutoff [sync](#)

0/4

P2 — AWS infra

DAY 1 · AUDIT & PLAN

Map the existing Step Functions state machine: list every state, its Lambda, its input/output shape

Identify exactly where Orchestrator output plugs in (before OCR) and where Critic plugs in (replacing QualityControl Lambda)

Create a new branch: infra/agent-pivot — all infra changes go here, keep main stable

Confirm Bedrock model availability in ap-south-1 for the model P1 is using — block P1 now if not [blocker](#)

Stand up a local Step Functions emulator (or small dev state machine) for P1 to test against without spending AWS credits

0/5

DAY 2 · WIRING

Create a new Lambda: modifai-orchestrator — calls P1's agent code, returns strategy JSON, fails loudly on bad output

Insert Orchestrator Lambda as the first state in the state machine — its output feeds into the OCR state's config

Create a new Lambda: modifai-critic — wraps P1's Critic batch function; reads samples from S3, writes verdicts back to S3

Replace the existing QualityControl Lambda with modifai-critic in the state machine [ship it](#)

End-to-end smoke test: trigger the pipeline with a small PDF, confirm it reaches the Critic state without errors

0/5

DAY 3 · RETRY LOOP

Add a Choice state after Critic: if $\text{accept\%} < \text{threshold}$ AND $\text{iteration} < 3$, branch to Curriculum state; else proceed to fine-tune

Create modifai-curriculum Lambda — calls P1's Curriculum agent, appends new samples to S3, re-triggers dataset generation

Add an iteration counter to the Step Functions execution context — must be visible in the execution history [ship it](#)

Test the retry loop: force low $\text{accept\%}$ (low threshold) and confirm it loops exactly 3 times then exits

Add a DynamoDB write at each loop end: iteration number, $\text{accept\%}$, reject count — P3 needs this for the live chart

DAY 4 · HARDENING

Write deploy.sh additions: create the 3 new Lambdas, update the state machine ARN — one command from scratch

Set Lambda timeouts correctly: Orchestrator 60s, Critic 5 min, Curriculum 3 min — prevent silent Step Functions failures

Run the full pipeline once in the real AWS account on the demo dataset — not the emulator blocker

Write infra section of README: what changed, how to deploy, environment variables required

0/4

P3 — Frontend

DAY 1 · DASHBOARD SCAFFOLD

Create a new route: /agent-run/:projectId — this is the live agent dashboard page

Build a skeleton AgentFeed component: a vertical timeline of agent decision events (use mock data today)

Each event card shows: agent name, timestamp, decision made, reason — styled with the person color system (purple=Orchestrator, teal=Critic, amber=Curriculum)

Add a status bar at the top: Current agent | Loop iteration N | Accept% so far

Redirect to /agent-run after pipeline start — replace the old 'pipeline running' loading screen

0/5

DAY 2 · LIVE FEED

Replace mock data with real polling: GET /api/v1/projects/:id/agent-events every 3 seconds

Ask P2 for the DynamoDB event format — build the API endpoint with P2 if needed sync

Animate new events sliding in from the bottom — use CSS transition only, no libraries

Add a Critic summary card: accept / rewrite / reject counts as three colored numbers, updates each loop

Handle the 'pipeline complete' state: stop polling, show final scores, surface a 'View dataset' button

0/5

DAY 3 · QUALITY GRAPH

Add a line chart below the event feed: X = loop iteration, Y = Critic accept% — use recharts or the existing chart lib

The chart must update live as new loop data arrives — each loop adds a new data point

Add a 'quality target' horizontal line at the threshold value set by the Orchestrator

Mobile check: the dashboard must be usable on a phone screen — judges may demo on mobile blocker

Add a 'Demo mode' URL param (?demo=1) that replays a pre-recorded run with fake 2-second delays — for the video

DAY 4 · DEMO MODE

Record the demo mode run — confirm it looks good as a screen recording before handing to P4

Polish pass: consistent spacing, no layout breaks on 1280px and 390px widths

Remove all console.log statements and TODO comments from frontend code — judges look at source

Verify the existing pipeline modes (Dataset Only, Fine-Tune Only etc.) still work — the agent route is additive, not a replacement

0/4

P4 — Demo & docs

DAY 1 · DEMO SCENARIO

Choose the demo dataset: 10–15 page PDF in a clear domain (customer support SOPs, HR policy, engineering runbook) — must produce interesting Critic rejections

Write a one-paragraph problem statement: who suffers, what they do today, why it's painful — this becomes the opening slide

Create the presentation deck structure: Problem → Current Modifai → Agentic pivot → Architecture → Live demo → Results → Ask

Research 2 real competitor tools (HuggingFace AutoTrain, AWS Jumpstart) — note what they can't do (no agentic loop, no curriculum)

Set up screen recording tool and test it — don't find out on Day 4 that audio isn't working **blocker**

0/5

DAY 2 · DECK CONTENT

Slide 1–2: Problem — use a concrete scenario, not abstract language. 'A company uploads 50 PDFs. Today they hand-tune thresholds. With Modifai, an agent decides.'

Slide 3–4: Architecture — use the diagram from the previous conversation; ask for an updated version if needed

Slide 5–6: The 3 agents — one slide per agent pair (Orchestrator+Critic, then Curriculum). One sentence each on what it does and why it matters.

Slide 7: Traction / differentiators — bullet the 3 things no other tool does (autonomous loop, curriculum top-up, no human threshold tuning)

Slide 8: Demo placeholder — leave a blank slide titled 'Live demo' — fill after Day 3 recording is done

0/5

DAY 3 · README & VIDEO

Rewrite the README completely: new project name/tagline if pivoting, then: What it does, The 3 agents (with P1's schema docs), Architecture, Setup, Demo

README must include a working 'Quick start' section — test it yourself from scratch in a clean directory **blocker**

Record the demo video: 2–4 minutes. Script: 30s problem, 30s architecture, 2min live dashboard walkthrough using P3's demo mode

Get P1 to narrate the agent decision section of the video — they can explain the Critic logic convincingly

Export video as MP4, upload to YouTube (unlisted) or Google Drive — confirm the link works from incognito

0/5

DAY 4 · SUBMISSION

Final GitHub check: repo is public, README renders correctly, no secrets or .env files committed **blocker**

Submission form: fill every field carefully — a blank 'tech stack' or 'team description' field is a red flag to judges

Add demo video link and live deployment link (if available) to the submission — judges should not have to ask for these

Submit at least 4 hours before the deadline — don't risk a last-minute AWS deploy failure costing you the slot **ship it**

0/4